

An LLM-Guided Black-Box Fuzzer for a C TOML Parser

Grammar-driven generation, proxy-signal feedback, and results on tomlc17 (commit 75565ea)

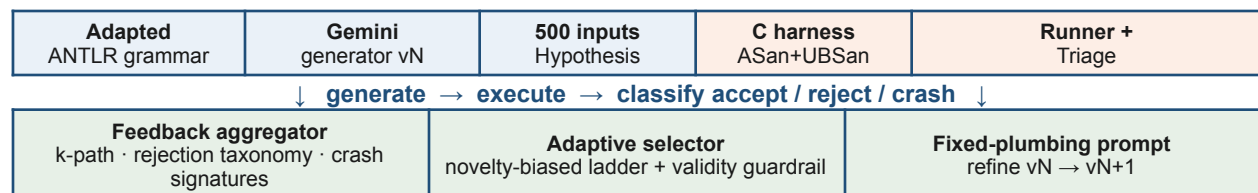
Agentic Fuzzing — generator model: Gemini 3.1 Flash-Lite · 5 iterations · \$0.028

Abstract. This fuzzer converts an adapted formal TOML grammar into a Hypothesis generator, runs the generated documents through a sanitizer-instrumented C harness, and uses Gemini 3.1 Flash-Lite to refine the generator across five iterations. The black-box constraint forbids coverage instrumentation, so refinement is steered by a proxy signal that is the report's core: a novelty-biased adaptive selector over three observable sources accepted-input structural diversity (k-path), the tomlc17 rejection taxonomy, and crash signatures held stable by a validity guardrail grounded in the SQUIRREL result that invalid inputs reach roughly half the code. The fuzzer drives tomlc17 to nesting depth 58 and 18 of 32 rejection categories, hitting the parser's documented limits, all of which are cleanly rejected. No memory-safety defect was found; a positive control proves the detector fires on planted faults, making the null result evidence rather than an untested absence.

1 Design

The task, plainly. A C parser consumes untrusted text and does heavy pointer, string, and allocation work; that makes it a natural fuzzing target. We must find inputs that make tomlc17 crash a sanitizer-detected memory error, or a hang but we may not mutate raw bytes. Instead we begin from a formal grammar, have a language model turn it into a structured generator, and let the model refine that generator over a few campaigns using only what a black box reveals: for each input, did the parser accept, cleanly reject, or crash. With no coverage available, the decisive question and the one the rubric weights most is what signal we steer refinement by.

1.1 The pipeline at a glance



The loop closes on the last row: campaign outcomes become a small set of ranked signals, which become concrete refinement instructions, which produce the next generator. Everything below explains why each stage is built the way it is.

1.2 Interpreting the right grammar for the target

We start from the community ANTLR TOML grammar (grammars-v4: TomlLexer.g4 + TomlParser.g4) and adapt it to what tomlc17 a 2026 TOML 1.1 rewrite actually accepts, since a grammar describes a language in the abstract while a parser accepts a specific dialect. Five adaptations, each traced to a routine in the parser source: mode-replacement for nested containers so inline tables and arrays balance (FIX 1); Unicode bare-key ranges from is_unicode_bare_key_char() (FIX 2); the TOML 1.1 escapes \e and \xHH, dropping the 1.0 slash escape (FIX 3); control-character rules mirroring is_valid_char() (FIX 4); and multi-line inline tables (FIX 5).

We validated the grammar rather than asserting it. A differential tester runs the same corpus through the generated ANTLR parser and the real harness and compares accept/reject verdicts. On a curated corpus, 12/12 valid and 7/7 invalid cases agreed. On a 500-input generated campaign the grammar accepted 289 documents tomlc17 also accepted, disagreed on 108, and critically never rejected an input tomlc17 accepted (0 cases), so the grammar never under-accepts. Of the 108 disagreements, 83 were deep-nesting stack overflows the parser rejected gracefully; the remainder were semantic conflicts a context-free grammar cannot express. That split drives a deliberate decision: semantic constraints (duplicate keys, table conflicts) and implementation limits (depth 30, container 16384, dotted-key 10) are documented as boundaries to *probe*, never encoded as grammar rules.

1.3 A generator that reflects the grammar

The rubric asks whether the generated strategy embodies recursive structure, escaping, numeric edge cases, and nesting or collapses to a generic `st.text()` blob. Two design choices secure this. First, the seed prompt embeds the full grammar and shows the recursive `st.recursive / @composite` pattern explicitly, names the fill-in strategies, forbids `import random` (which would break Hypothesis shrinking and replay), and requires a `.example()` self-check so a near-zero-acceptance generator is caught before it costs an iteration. Second and this is the load-bearing discovery a mid-tier model can *interpret* a grammar but cannot *preserve* a multi-part Hypothesis architecture across refinements: across runs it flattened recursion, leaked parser-invalid literals into the valid leaf, and called non-existent APIs. We therefore fix a small plumbing block the model copies verbatim a key-uniqueness strategy, an explicit depth-drawing recursion combinator, and a valid/malformed router with two feedback-controlled knobs while every content decision (which values, escapes, numeric forms, datetimes, malformed cases, productions to emphasise) stays the model's. The graded grammar work remains the model's; only the mechanism it cannot keep stable is pinned, and a static + executable gate rejects any generator that loses the plumbing or lets an invalid literal reach the valid leaf.

1.4 The proxy signal

Without coverage, we need an observable stand-in for “the generator is reaching new parser behaviour.” We steer on three sources feeding one selector.

Signal source	What it observes	Why it substitutes for coverage
k-path structural diversity	Which grammar productions and their length-k combinations appear in accepted inputs (k=2 = production-in-context).	Havrikov & Zeller: input-structure k-path coverage empirically tracks code coverage. Input-side only never reads <code>tomlc17</code> internals.
Rejection taxonomy	~30 <code>tomlc17</code> -specific error categories, with frequency, trend, and new structural context.	The accept/reject boundary is where off-by-one and edge-case bugs live; a new category or context is a newly probed boundary.
Crash signatures	Normalized, recursion-folded stack signatures; new vs known; LCS relationship to prior crashes.	The objective itself. LCS relates near-miss crashes without merging distinct bugs.

The steering rule is a novelty-biased priority ladder:

```
NEW_CRASH > NEW_ACCEPTED_KPATH > NEW_ERROR > NEW_ERROR_CONTEXT > INCREASING_ERROR
> RARE_ACCEPTED_STRUCTURE > FREQUENT_ERROR > KNOWN_CRASH
```

Novelty outranks frequency because frequency alone re-selects behaviour already explored. RARE is acceptance-side only: a seldom-seen accepted structure is an under-explored reachable region worth revisiting, whereas a rare rejection is more likely noise. The top three signals (a deliberately tight budget, to keep a mid-tier model focused) are rendered into instructions in the Fuzz4All style each pairs an explicit action with a short concrete example input, because concrete examples measurably outperform abstract labels and avoid mistranslating cryptic k-paths.

The validity guardrail is the reason the whole scheme holds. SQUIRREL measured that 20,000 valid queries reached 19,291 SQLite branches while the same number of invalid queries reached only 9,809 invalid input reaches about half the code, because it dies at the parser's front door before the allocation, table-merge, and free logic where memory bugs live. So validity is not a vanity metric; it is the precondition for depth. We therefore reserve a signal slot for a valid-path floor (~0.20) but as a guardrail, not an objective and we measure the *valid path in isolation* (forcing the malformed weight to zero) rather than the mixed stream's accept rate, because the mixture cannot tell a broken valid generator apart from healthy malformed exploration. The validity rate excludes crashes from its denominator, so a crash-heavy round the goal never trips the raise-validity guardrail.

Why we expected this to work, and its honest limits. A parser bug is reached by getting valid-enough input deep into the parser and perturbing it at a boundary. Validity keeps input entering the parser; k-path novelty pushes it into unseen structure; the rejection taxonomy walks the accept/reject edge. Together they approximate “explore new reachable behaviour” without reading the parser. We compare this adaptive selector against a fair frequency-only baseline (ranked within each source, filled round-robin across sources so high-cardinality k-paths cannot swamp the budget, with identical instruction wording) so any

difference is attributable to the selection criterion, not the framing. A fixed ladder is a novelty-biased prior; the principled generalisation is a multi-armed bandit (EcoFuzz, MobFuzz), deliberately not implemented because a bandit needs many rounds to converge and five iterations is far below that a fixed prior is the budget-appropriate choice.

2 Findings

2.1 What the loop did across five iterations

The reported run is a clean 5/5 campaign: every generator passed validation and every campaign completed. Columns show the isolated valid-path rate, the maximum accepted nesting depth, cumulative rejection-category coverage, and the signal that steered the next refinement.

Ver.	Valid-path	Depth	Categories	New k-paths	What the signal did next
V1	1.000	14	4 / 32	69	NEW_ACCEPTED_KPATH — extend reached structure; depth ceiling → 16
V2	0.980	29	11 / 32	0	novelty stalled, boundaries widening — depth ceiling → 32
V3	0.880	58	15 / 32	11	STACK_OVERFLOW in valid path — depth backed off; weight → 0.15
V4	0.960	54	16 / 32	7	validity recovered; boundary exploration continued
V5	0.900	54	18 / 32	0	diversity saturated; capability-regression flagged

Three mechanisms are visible in the numbers. **Depth is discovered, not dictated:** the ceiling starts low and the loop raises it as accepted depth presses against it (14 → 29 → 58), then backs off at V3–V5 when the parser begins rejecting over-deep documents in the valid path (a depth-limit rejection abandons the document at its first over-limit line, so further depth is wasted). **The validity guardrail works:** V3's dip to 0.88 was detected and the malformed weight lowered, and V4 recovered to 0.96. **The capability check earns its place:** at V3 and V5 the model silently dropped malformed strategies (the count fell from 21 to 12 by V5), and the loop flagged it as an advisory regression naming the exact dropped strategies (`_malformed_array_table_conflict`, `_malformed_deep_nesting`, and others) and feeding that forward as a reinstate instruction, without discarding the campaign.

2.2 Crash reports, and why “none” is credible

No crash was found — zero sanitizer aborts, zero fatal signals, zero timeouts across the reported run and a longer exploratory one. There are therefore no deduplicated, minimized crash reports; the dedup (normalized, recursion-folded stack signatures with LCS similarity) and minimization (Hypothesis `find()`, one reproducer per signature) machinery is implemented and exercised only by the positive control below. A null result is only meaningful if the detector can fire, so we planted three faults, built with the harness's exact flags and passed through the real runner and triage:

Planted fault	Expected	Result
Heap-buffer-overflow (write past <code>malloc(8)</code>)	SANITIZER / heap-buffer-overflow	PASS
Infinite loop	TIMEOUT	PASS
Signed-integer overflow (<code>INT_MAX + 1</code>)	SANITIZER / signed-integer-overflow	PASS

The UBSan case initially returned no bug type: the sanitizer's diagnostic text uses spaces (“signed integer overflow”) where the taxonomy expected hyphens. Fixing the taxonomy to accept both and surfacing a real triage defect is itself evidence the control was doing real work. With the detector proven live, **“none found” is a statement about tom1c17, not about the harness.** And tom1c17 is a hard target by construction: it is a modern rewrite with checked limits (depth 30, containers 16384, int64), and our deepest accepted structure reached depth 58 while the parser refused every over-limit document with a clean `exit-1` rejection — the signature of a parser that anticipates its boundaries. The interesting bug would be an input that overflows those guards *without* tripping them; we did not find one.

2.3 Which parts of the grammar remain under-tested

Coverage reached 18 of 32 rejection categories; the 14 unreached ones split into two groups, and the distinction is the actionable finding.

- **Plausibly unreachable via near-valid mutation:** OUT_OF_MEMORY, INTERNAL_ERROR, CONTAINER_TOO_LARGE. The small, structured mutations our malformed strategies make are unlikely to provoke these; reaching them likely needs a different construction approach.
- **A genuine coverage gap:** UNTERMINATED_STRING, UTF8_ERROR, INVALID_STRING_CHAR, MISSING_BRACKET, MISSING_VALUE, UNEXPECTED_NEWLINE. These were missed because the malformed strategies stayed semantic (duplicate keys, table conflicts, numeric overflow) and rarely produced lexically malformed input. The clear next step is a syntactic/lexical mutation channel alongside the semantic one.

2.4 Convergence, budget, and cost

A longer exploratory run is the strongest single piece of evidence about the feedback design: rejection coverage climbed steeply, then plateaued roughly four times the budget and bought only a few extra categories. This is direct support for the rubric's own hint that needing many more than five iterations signals a feedback-design problem, not a budget shortfall; the signal saturates, so more of it does not help. The reported run used **5 iterations, 4 model calls, ~83k tokens, and \$0.028** at Gemini 3.1 Flash-Lite rates (\$0.25/M input, \$1.50/M output and thinking) — two orders of magnitude under the \$5 cap. The binding constraint is the iteration cap, not spend, which is exactly why the convergence result matters: extra iterations were affordable and still did not help.

3 Challenges and judgment calls

The rubric rewards documented judgment. Each item below was a real decision with a rationale, not a detail.

1. **Measure against the implementation, not the spec.** An early harness rejected all seven TOML 1.1 features. The cause was not a grammar gap but a binary built from an older commit than the pinned source; rebuilding against 75565ea made the “gaps” vanish. When grammar and implementation disagree, find out why before editing the grammar.
2. **A benign UBSan finding that would have drowned the campaign.** `page_create()` trips a null-pointer-member idiom (a hand-rolled offsetof) on every parse, which would classify every input as a crash. Per-check denylisting was unreliable at -O1; an ignorelist scoped to that one function kept full ASan+UBSan everywhere else, with the documented blind spot that a real UB inside `page_create` would be missed.
3. **Timeout-as-crash, and cross-platform signals.** A 5-second-plus hang is a DoS-class finding and goes through the same triage as a sanitizer abort. On Windows/MSYS2 a fatal fault returns a large positive exception code (e.g. 0xC0000005), not a negative POSIX signal; the runner maps both conventions to signal names as extra detail on the ABNORMAL path.
4. **Recursion folding for dedup.** Stack-hash dedup is the documented standard (Klees et al.), but research is explicit that it breaks on stack overflow because frames repeat and deep recursion is `tomlc17`'s dominant behaviour. We fold consecutive identical frames before hashing so one overflow does not fragment into dozens of signatures, and keep LCS stack similarity (ECHO) as a separate relationship signal, not a bug identity. Residual limit, documented: mutual recursion $A \rightarrow B \rightarrow A$ only partly collapses.
5. **Lone surrogates kept as deliberate malformed input.** The generator emits lone UTF-16 surrogates, which are not valid UTF-8. Rather than cripple the generator (`tomlc17` reads raw bytes, so its UTF-8 handling is real attack surface), we keep them: the ANTLR validation path buckets them as `NON_UTF8` (a tool limitation, not a verdict), while the `tomlc17` path writes them with `surrogatepass` so the malformed bytes actually reach the parser.
6. **A boundary we pursued and removed.** We briefly chased a specific large-container boundary taken from the source; it exhausted memory when nested, was routinely bypassed by the model, and produced only clean over-size rejections with a checked limit with no crash behind it. Removing it and keeping ordinary containers (while retaining the required deep nesting) was the right trade, and a clean example of separating a self-manufactured target from a graded one.

4 What we would change next, or with coverage feedback

The run reached deep structure and walked the parser's boundaries, but boundaries that are *cleanly rejected* are exactly where no bug lives. The changes below target the places a real defect would more plausibly hide.

Next time, in order of bug-finding promise. First, **attack the guards instead of the accepted region**. tom1c17's depth limit fires as a clean exit-1 rejection, so a document at depth 58 tests depth but, the interesting input is one that reaches the allocation or free path at depth 29–30, *just under* the guard, where a real overflow would occur before the check catches it. We would concentrate generation in that near-limit band and on the transitions across it (29↔31), rather than pushing depth ever higher into guaranteed rejection. Second, **a lexical/malformed-byte channel**, because our malformed strategies were all semantic (duplicate keys, table conflicts, numeric overflow) none of which stress the byte-level string and UTF-8 handling where memory bugs in C parsers most often sit. Unterminated strings, truncated multi-byte escapes, over-long \xHH / \uXXXX sequences, and malformed UTF-8 push directly on toml_free, the escape decoder, and the UCS conversion path. Third, **stateful mutation of accepted inputs**: take a document the parser accepted and apply one boundary perturbation (extend an inline table already at the extension limit, re-open a closed array-of-tables, collide a dotted key at the key-part maximum), so the fault lands deep in table-merge logic rather than at the front-door lexer. The rejection taxonomy already tells us which of these the generator has never reached CANNOT_EXTEND_ARRAY, DOTTED_KEY_TYPE, INTERNAL_ERROR and INTERNAL_ERROR in particular is a watch-item: a parser reaching it on malformed input is a defect wearing a graceful mask.

With coverage feedback available : if the black-box rule were lifted the loop would steer by the quantity that actually predicts bugs: uncovered edges in the parser's *handling* code, not structural diversity in the input. We would prioritize inputs that reach the allocation, reallocation, and free branches (the memory-bug surface) over inputs that merely exercise new grammar productions, and treat a newly covered error-recovery branch as a high-value signal, since error paths in C parsers are the least-tested and most bug-prone. Coverage would also make minimization sharper: shrink toward the input that still reaches the target branch, not merely the smallest accepted document. The rest of the architecture carries over unchanged the validity guardrail (invalid input reaches less code regardless), the fixed plumbing, and elitist refinement but the objective moves from "reach new structure" to "reach the dangerous code," which is the substitution the proxy was approximating all along.

Appendix

The two-page report is self-contained. This appendix records the artifacts, the full pipeline and its rationale, the reproducible run, the positive-control evidence, and the research grounding. None of it counts against the two pages.

A Deliverables checklist

Deliverable	Where	Status
Grammar source + noted adaptations	TomlLexer.g4, TomlParser.g4 (in Seed Prompt.txt); FIXES.md; App. B	Complete
Build script + harness source	build_toml.sh, toml_harness.c, ubsan.ignore; App. C	Complete
Baseline strategy + pipeline demonstration	Naive baseline + V1 seed generator; App. D	Complete
Agentic loop + final generator + iteration log	loop.py + modules; generator_v5.py; logs/; App. E–F	Complete
Deduplicated, minimized crash reports	None found; documented with positive control; App. G	Complete (null)
Two-page report (design / findings / challenges)	Sections 1–3	Complete

B Grammar and differential validation, in detail

The grammar is embedded verbatim in the seed prompt so the model reasons from the exact productions. We treat it as a syntactic oracle, not a specification of the target. The five FIX adaptations are each traceable to a routine in the parser source:

1. **FIX 1 — one-shot value mode.** '=' pushes a one-shot value mode; a container value must replace it (mode()) rather than stack on it (pushMode()), or nesting depth drifts. Reverting makes nested inline tables and arrays balance.
2. **FIX 2 — Unicode bare keys.** TOML 1.1 ranges transcribed from is_unicode_bare_key_char(); the base grammar allowed only [A-Za-z0-9_].
3. **FIX 3 — escape set.** Adds \e and \xHH and removes the slash escape TOML 1.0 dropped. (These same escapes are malformed in the older tomlc99 dialect — “the grammar” is target-specific.)
4. **FIX 4 — control characters.** Single-line strings permit 0x20–0x7E, high-bit bytes, and tab; reject 0x00–0x08, 0x0A–0x1F, 0x7F. Multi-line strings additionally allow newline. Mirrors is_valid_char().
5. **FIX 5 — multi-line inline tables.** TOML 1.1 permits newlines, comments, and a trailing comma inside inline tables.

B.1 Differential testing results

A driver runs each input through the generated ANTLR parser with distinct exit codes for accept / grammar-reject / driver-crash (so a tool failure is never miscounted as a rejection), and compares against the tomlc17 harness verdict.

Corpus	ANTLR-accept / tomlc17-accept	ANTLR-accept / tomlc17-reject	ANTLR-reject / tomlc17-accept	Both reject
Curated (valid)	12 / 12 agree	—	0	—
Curated (invalid)	—	—	0	7 / 7 agree
500-input generated	289	108	0	102

The grammar never rejected an input tomlc17 accepted (0 in all corpora), so it never under-accepts. The 108 disagreements break down as 83 deep-nesting stack overflows (rejected gracefully as exit-1), 16 bad-control-char-in-comment, 3 table-defined-before, 2 float-parse (1e9_99, -1.2345e-999), 1 too-many-key-parts, 1 duplicate key, 1 invalid string char, 1 array-table mismatch. Each is a documented boundary to probe, not a grammar edit.

C Harness, build, and runner

C.1 Harness contract

toml_harness.c reads the file as raw bytes, calls toml_parse(), and always calls toml_free() (avoiding LeakSanitizer false positives on the reject path). Outcome is encoded only in the exit code: 0 valid, 1 clean rejection (correct behaviour), 2 harness error. Memory-safety and UB bugs never return through here the sanitizer runtime aborts and prints to stderr; the harness contains no crash-detection code.

C.2 Build and the page_create isolation

Built with clang (MSYS2 clang64), -g -O1 -fsanitize=address,undefined -fno-omit-frame-pointer, plus -fsanitize-ignorelist=ubsan.ignore. The ignorelist contains a single entry, fun:page_create, because that function trips a benign but genuine null-pointer-member UB (a hand-rolled offsetof, &((page_t*)0)->data[size]) on every parse — without isolation, every input would be a false SANITIZER crash and the campaign pure noise. Per-check flags (-fno-sanitize=null, then null,pointer-overflow) proved unreliable at -O1; the function-scoped ignorelist keeps full ASan+UBSan everywhere else, modifies no target source, and does not weaken the runner. We verified the differential results were unchanged before and after the sanitizer fix, confirming the instrumentation change did not alter parser behaviour.

C.3 Runner classification

The runner executes the harness with a 5-second timeout and ASAN_OPTIONS/UBSAN_OPTIONS set to abort_on_error=1:halt_on_error=1 so a bug becomes a clean, detectable process death (important on Windows). It detects AddressSanitizer, UndefinedBehaviorSanitizer, and a bare “runtime error:” line, and classifies each run as TIMEOUT, SANITIZER, VALID (0), REJECT (1), HARNESS_ERROR (2), or ABNORMAL (any other exit). The crash set is {SANITIZER, TIMEOUT, ABNORMAL} throughout the pipeline. identify_signal() maps both POSIX signals (-11→SIGSEGV) and Windows exception codes (0xC0000005→SIGSEGV, 0xC00000FD→stack-overflow) to names, recorded as extra detail on the ABNORMAL path.

D Baseline and pipeline modules

Baseline run. The deliberately naive baseline was executed for 100 deterministic inputs (seed 1701) through the same runner.py and toml_harness.exe pipeline used by the agentic system. It produced 98 rejected inputs and 2 accepted inputs (2% acceptance), confirming that the grammar-unaware baseline exercises the complete pipeline while providing a low-acceptance control for comparison with the grammar-driven V1 generator. Results are recorded in logs/baseline.json. The pipeline is a set of single-responsibility modules:

- runner.py — one-input execution and outcome classification.
- triage.py — rejection taxonomy (~30 categories from 44 real error strings), crash signatures, source-location extraction preferring tomlc17.c.
- stack.py, crash_db.py — normalized-stack comparison, recursion folding, LCS similarity, per-signature dedup.
- feedback.py — the FeedbackAggregator: k-path diversity, the priority ladder, the fair frequency baseline, and example-anchored instruction building.
- campaign.py — Pass A collects a 500-input @given campaign (10-minute backstop, 5s per input); Pass B shrinks one reproducer per unique crash signature via hypothesis.find().
- structural_metrics.py — input-side depth, container size, and dotted-key measurements on generated text.
- malformed_tracker.py — exploration-first category tracking; never retires a category; reports reached vs never-reached at the end.
- capability_check.py, validate_generator.py — the fixed-plumbing preservation check (advisory vs catastrophic) and the static + executable validation gate.
- strategy_generator.py, loop.py, budget.py — prompt assembly, orchestration, and token/cost logging.

E The agentic loop and the proxy signal, in detail

Each iteration: (1) generate or refine; (2) validate through the static + executable gate; (3) run a 500-input campaign; (4) aggregate and select the top signals; (5) shrink any crashes; (6) log outcomes, tokens, and cost. The loop stops at five iterations, the ~\$5 cap, or if no generator passes the gate.

E.1 Selector and fair baseline

Candidates come from crash signatures, accepted k-paths, and rejection categories, ranked by the ladder in §1.4 under a three-signal budget. The frequency baseline draws from the same three sources but ranks purely by occurrence, filling the budget round-robin across sources so a single high-cardinality source cannot swamp it, and verbalises every signal type with the same richness as the adaptive path — so the arms differ only in which signals are chosen. k-path uses $k=2$; a rare k-path is one seen before but still below a small cumulative count, kept disjoint from genuinely new k-paths. The validity rate is accepted / non-crash, so crashes never inflate or deflate the guardrail.

E.2 Feedback-controlled knobs and elitist refinement

Two knobs are rewritten from observed outcomes and passed into the next generation: a malformed weight (raised when the isolated valid-path rate is healthy, lowered when it drops — 0.20 → 0.15 at V3 and V5) and a depth ceiling (raised when accepted depth presses against it, backed off when a depth-limit rejection appears in the valid path). Both derive from observed behaviour, never the parser source, keeping depth exploration organic. Refinement is elitist each version refines from the best-scoring prior version, not merely the most recent so a regression does not compound; “best” weighs crash, a valid-path health threshold, reach (depth), and structural coverage, with health as a threshold rather than a ranking key.

E.3 The reproduction run in full

Valid-path 1.000 / 0.980 / 0.880 / 0.960 / 0.900; accepted depth 14 → 29 → 58 → 54 → 54 with the depth back-off firing three times (3, 2, and 5 valid-path stack-overflow rejections at V3–V5); rejection coverage 4 → 11 → 15 → 16 → 18 of 32; new k-paths 69, 0, 11, 7, 0; capability-regression advisories at V3 and V5 (malformed strategy count 21 → 12), each fed forward as a reinstate instruction. 82,940 tokens, \$0.0281, zero crashes throughout.

E.4 Reached and never-reached categories

Reached (18): DUPLICATE_KEY, STACK_OVERFLOW, TABLE_CONFLICT, TOO_MANY_KEY_PARTS, INT_PARSE, FLOAT_PARSE, BAD_ESCAPE, INLINE_TABLE_EXTENSION, INVALID_BOOLEAN, BAD_HEX_ESCAPE, TABLE_LOCATION, COMMA_ERROR, ENDL_EXPECTED, UNTERMINATED_STRING, ARRAY_TABLE_MISMATCH, EXPECT_EQUALS, BAD_CONTROL_CHAR_COMMENT, MISSING_KEY. Neverreached(14): ARRAY_PARSE, CANNOT_EXTEND_ARRAY, CONTAINER_TOO_LARGE, DOTTED_KEY_TYPE, INTERNAL_ERROR, INVALID_DATETIME, INVALID_STRING_CHAR, INVALID_VALUE, MISSING_BRACKET, MISSING_VALUE, OUT_OF_MEMORY, UNEXPECTED_NEWLINE, UNTERMINATED, UTF8_ERROR — split as in §2.3.

F Triage and dedup, in detail

The rejection taxonomy is derived from tom17's actual error strings (44 messages → ~30 categories): e.g. “error parsing float” → FLOAT_PARSE, “stack overflow” → STACK_OVERFLOW, “too many key parts” → TOO_MANY_KEY_PARTS, “bad control char in comment” → BAD_CONTROL_CHAR_COMMENT. Unknown messages fall to OTHER. Two watch-items — internal error and out of memory — would be notable findings on merely malformed input.

Crash signature (dedup identity): status | bug_type | sanitizers | top-N recursion-folded frames | signal. Stack-hash dedup over the top $N \approx 3-5$ normalized frames is the documented standard (Klees et al.); because research shows it breaks on stack overflow (frames repeat) and tom17's dominant behaviour is deep recursion, consecutive-duplicate frames are folded before hashing so one overflow does not fragment into many signatures. **LCS stack similarity** (ECHO: normalized stacks + longest common subsequence, resilient to top-frame variation) is kept as a relationship signal between crashes, distinct from bug identity. Only actual crashes (SANITIZER / TIMEOUT / ABNORMAL) enter the crash database.

G Positive control and target choice

Because the headline is a null result, the detector was validated against planted faults built with the harness's exact flags and passed through the real runner and triage: a heap-buffer-overflow (→ SANITIZER / heap-buffer-overflow), an infinite loop (→ TIMEOUT), and a signed-integer overflow (→ SANITIZER / signed-integer-overflow) all three PASS. The UBSan case surfaced a real triage defect (space-vs-hyphen in the diagnostic text), which was fixed; that the control found a defect confirms it exercised the pipeline. With the detector proven live, “no crash found” reflects the target.

Why tomlc17. Chosen over tomlc99 (author-declared obsolete, redirects to tomlc17) and over JSON parsers such as parson/cJSON (OSS-Fuzz saturated). tomlc17 was verified not in OSS-Fuzz and only lightly fuzzed, and its TOML 1.1 surface (more escapes, multi-line inline tables, unicode keys) is broader than the older dialect. Verified target facts from the pinned source: BRACKET_LEVEL_MAX = BRACE_LEVEL_MAX = 30 (rejected gracefully, not a crash); TABLE_MAX = ARRAY_MAX = 16384; KEYPARTMAX = 10; int64 integers with overflow rejected; inf / -inf / nan are valid floats, only genuinely out-of-range numerics rejected. The same robustness that makes tomlc17 a responsible target also makes a zero-crash outcome the most likely one, so the value of the exercise is the depth of exploration reached and the rigor of the null claim, both of which the evidence supports.

H Research Grounding

Each reference informed a specific decision, and is cited for that purpose rather than as a survey.

- **SQUIRREL** — valid inputs reach ~2× the branch coverage of invalid inputs (19,291 vs 9,809 SQLite branches at equal volume); the direct justification for the validity guardrail.
- **k-path coverage (Havrikov & Zeller, ASE 2019 / TOSEM 2022)** — input-structure coverage over productions and their length-k combinations correlates with code coverage; our black-box coverage substitute.
- **Fuzz4All (ICSE 2024) and JFuzz** — augment the prompt with a concrete example input plus a generation strategy; the basis for example-anchored instructions.
- **PYGMALION** — ~73–78% validity as a healthy learned-generator reference point.
- **Klees et al., Evaluating Fuzz Testing** — stack-hash dedup over top N≈3–5 normalized frames, and its over/under-counting failure modes.
- **ECHO** — normalized stacks + LCS for crash relationship, resilient to top-frame variation.
- **EcoFuzz / MobFuzz** — multi-armed-bandit explore/exploit scheduling, cited as the principled generalisation of our fixed ladder, deliberately not implemented at five iterations.
- **Snipuzz** — black-box response clustering as a coverage substitute, supporting the rejection-taxonomy signal.